Moragora API 명세서

Express + Supabase + Socket.io + Multi-AI Judgment System — 토론 판결 플랫폼 백엔드 API

목차

1. 시스템 개요
2. 인증 방식
3. 데이터베이스 스키마 (16 tables)
4. 인증 API (/api/auth)
5. 토론 API (/api/debates)
6. 주장 API (/api/arguments)
7. 판결 API (/api/judgments)
8. 투표 API (/api/votes)
9. 프로필 API (/api/profiles)
10. 콘텐츠 필터 API (/api/content)
11. AI 판결 시스템
12. 피드백 API (/api/feedbacks)
13. OG 메타 API (/og)
14. 채팅 API (/api/chat) [NEW]
15. 댓글 API (/api/comments) [NEW]
16. 알림 API (/api/notifications) [NEW]
17. AI 생성 API (/api/ai) [NEW]
18. Cron API (/api/cron) [NEW]
19. 관리자 API (/api/admin) [NEW]
20. Socket.io 이벤트 [NEW]

21. 에러 코드 정리

1. 시스템 개요 ▼

```
┌──────────────────────────────────────────────────────────────────┐
│                        Client (React + Vite)                      │
│                     Vercel: team-moragora-client                  │
└──────────────────────────────────────────────────────────────────┘
                │                                │
                │  HTTPS (axios + JWT)           │  WebSocket (Socket.io)
                ▼                                ▼
┌──────────────────────────────────────────────────────────────────┐
│                       Express Server (Render)                     │
│  Port: 5000  │  CORS: CLIENT_URL  │  Base: /api                  │
├──────────────────────────────────────────────────────────────────┤
│  Middleware: requireAuth │ contentFilter │ errorHandler            │
├──────────────────────────────────────────────────────────────────┤
│                     Socket.io (Real-time Layer)                   │
│  join-room │ send-message │ typing │ presence │ game-start        │
├──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────┤
│  /auth   │/debates  │/arguments│ /judgments │ /votes  │/prof.    │ /content │
├──────────┼──────────┼──────────┼──────────┼──────────┼──────────┼──────────┤
│  /chat   │/comment  │/notify   │ /ai      │ /cron    │/admin    │/feedbacks│
└──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┘
     │         │         │         │         │         │         │
     ▼         ▼         ▼         ▼         ▼         ▼         ▼
┌──────────────────────────────────────────────────────────────────┐
│                     Supabase (PostgreSQL + Auth)                  │
│  Tables (16): profiles, debates, arguments, verdicts, ai_judgments,
│  votes, content_filter_logs, xp_logs, feedbacks, comments,        │
│  chat_messages, notifications, debate_likes, verdict_ratings,     │
│  page_views, analytics_events                                     │
└──────────────────────────────────────────────────────────────────┘
                          │
                ┌─────────┴─────────┐
                ▼         ▼         ▼
            ┌─────────┐ ┌─────────┐ ┌─────────┐
            │GPT-4.1  │ │Gemini2.5│ │ Claude  │   ← 3개 AI 병렬 판결
            │Judge G  │ │ Judge M │ │Judge C  │
            └─────────┘ └─────────┘ └─────────┘
                │         │         │
                └─────────┴─────────┘
                          ▼
                      ┌─────────┐
                      │ Grok 3 Mini│  ← 실패 시 대체 판사
                      │ Judge X  │
                      └─────────┘
```

항목	내용
서버	Express.js (Node.js) — Render 배포
데이터베이스	Supabase (PostgreSQL + Auth + RLS) — 16 tables
실시간	Socket.io (WebSocket) — 채팅, 프레즌스, 타이핑 표시

프론트엔드	React + Vite — Vercel 배포
인증	Supabase Auth (JWT Bearer Token)
AI 판결	GPT-4.1, Gemini 2.5 Flash, Claude Sonnet 4 (병렬) + Grok 3 Mini (폴백)
콘텐츠 필터	3단계 (사전 필터 → AI 유해성 → AI 주제 적합성)
입력 전처리	프롬프트 인젝션 방어 + 텍스트 정규화 + 유사도 검증

2. 인증 방식

Supabase Auth 기반 JWT 토큰 인증을 사용합니다.
Request Header

```
Authorization: Bearer <supabase_access_token>
```

인증 미들웨어 (requireAuth)

동작	설명
토큰 검증	Supabase Admin으로 JWT 검증 후 사용자 정보 추출
req.user 설정	검증된 사용자 객체를 `req.user` 에 바인딩
실패 시	`401` Unauthorized 반환

> **참고:** 공개 API(토론 목록, 판결 피드, 랭킹 등)는 인증 없이 접근 가능합니다. 각 엔드포인트의 `AUTH` / `PUBLIC` 뱃지를 확인하세요.

3. 데이터베이스 스키마 (16 tables)

PK profiles

컬럼	타입	제약조건	설명
id	UUID	PK, FK auth.users CASCADE	사용자 ID
nickname	TEXT	NOT NULL	닉네임
gender	ENUM	male/female/other, NULL	성별
age	INT	1~150, NULL	나이
avatar_url	TEXT	NULL	프로필 이미지 URL
wins	INT	DEFAULT 0	승리 수
losses	INT	DEFAULT 0	패배 수
draws	INT	DEFAULT 0	무승부 수
total_score	INT	DEFAULT 0	총 점수
xp	INT	DEFAULT 0	경험치
tier	ENUM	시민/배심원/변호사/판사/대법관	등급 (DEFAULT 시민)
created_at	TIMESTAMPTZ	DEFAULT NOW()	가입일

PK debates

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	토론 ID
creator_id	UUID	FK profiles, NOT NULL	생성자
opponent_id	UUID	FK profiles, NULL	상대방 (참여 시 설정)
topic	TEXT	NOT NULL	토론 주제
description	TEXT	NULL	토론 설명 (선택)
category	TEXT	DEFAULT 'daily'	카테고리
purpose	TEXT	DEFAULT 'compete'	판결 목적 (compete/consensus/analysis)
lens	TEXT	DEFAULT 'general'	분석 렌즈 (logic/emotion/practical/ethics/general)
mode	TEXT	NOT NULL, DEFAULT 'duo', CHECK ('duo','solo','chat')	토론 모드 (duo: 대인, solo: AI 상대, chat: 실시간 채팅)
invite_code	TEXT	UNIQUE, NOT NULL	초대 코드 (8자)
status	TEXT	NOT NULL	waiting → chatting → arguing → judging → voting → completed
vote_deadline	TIMESTAMPTZ	NULL	투표 마감 시간
view_count	INT	DEFAULT 0	조회수
created_at	TIMESTAMPTZ	DEFAULT NOW()	생성일

PK arguments

컬럼	타입	제약조건	설명
id	UUID	PK	주장 ID
debate_id	UUID	FK debates, NOT NULL	토론 ID
user_id	UUID	FK profiles, NOT NULL	작성자 (Solo 모드: NULL)
side	TEXT	'A' 또는 'B'	진영
content	TEXT	50~2,000자	주장 내용 (전처리 완료)
ai_generated	BOOLEAN	DEFAULT false	AI 생성 여부 (Solo 모드)
created_at	TIMESTAMPTZ	DEFAULT NOW()	작성일

PK verdicts

컬럼	타입	제약조건	설명
id	UUID	PK	판결 ID
debate_id	UUID	FK debates, UNIQUE	토론 ID (1:1)
winner_side	TEXT	'A' / 'B' / 'draw'	최종 승자
summary	TEXT		판결 요약
ai_score_a	INT	DEFAULT 0	AI 종합 점수 A
ai_score_b	INT	DEFAULT 0	AI 종합 점수 B
citizen_score_a	INT	NULL	시민 투표 점수 A
citizen_score_b	INT	NULL	시민 투표 점수 B
final_score_a	INT		최종 점수 A (AI 75% + 시민 25%)
final_score_b	INT		최종 점수 B
is_citizen_applied	BOOLEAN	DEFAULT false	시민 투표 반영 여부
created_at	TIMESTAMPTZ	DEFAULT NOW()	판결일

PK ai_judgments

컬럼	타입	제약조건	설명
id	UUID	PK	AI 판결 ID
verdict_id	UUID	FK verdicts	판결 ID
ai_model	TEXT		gpt-4.1 / gemini-2.5-flash / claude-sonnet / grok-3-mini
winner_side	TEXT	'A' / 'B' / 'draw'	개별 AI 판정
verdict_text	TEXT		판결문 (캐릭터별 톤)
score_a	INT	0~100	A측 총점
score_b	INT	0~100	B측 총점
score_detail_a	JSONB		{logic, evidence, persuasion, consistency, expression}
score_detail_b	JSONB		{logic, evidence, persuasion, consistency, expression}
confidence	NUMERIC	0.00~1.00	판결 확신도
created_at	TIMESTAMPTZ	DEFAULT NOW()	판결일

PK votes

컬럼	타입	제약조건	설명
id	UUID	PK	투표 ID
debate_id	UUID	FK debates	토론 ID
user_id	UUID	FK profiles	투표자
voted_side	TEXT	'A' 또는 'B'	투표한 진영
created_at	TIMESTAMPTZ	DEFAULT NOW()	투표일

PK `xp_logs`

컬럼	타입	제약조건	설명
id	UUID	PK	로그 ID
user_id	UUID	FK profiles	사용자
xp_amount	INT	NOT NULL	XP 변동량
reason	TEXT		victory/defeat/draw/daily_debate/vote/bonus/penalty
reference_id	UUID	NULL	관련 토론 ID
created_at	TIMESTAMPTZ	DEFAULT NOW()	기록일

PK `content_filter_logs`

컬럼	타입	제약조건	설명
id	UUID	PK	로그 ID
user_id	UUID	FK profiles	사용자
content_type	TEXT		콘텐츠 유형
filter_stage	INT	1 / 2 / 3	필터 단계
blocked	BOOLEAN		차단 여부
reason	TEXT	NULL	차단 사유
created_at	TIMESTAMPTZ	DEFAULT NOW()	기록일

PK feedbacks

컬럼	타입	제약조건	설명
id	UUID	PK	피드백 ID
user_id	UUID	FK profiles, UNIQUE	사용자 (1인 1피드백)
satisfaction	NUMERIC	0.5~5.0	만족도
ai_accuracy	NUMERIC	0.5~5.0	AI 정확도
ui_ease	NUMERIC	0.5~5.0	UI 편의성
fairness	NUMERIC	0.5~5.0	공정성
recommend	NUMERIC	0.5~5.0	추천 의향
best_feature	TEXT	NULL	최고 기능
improvement	TEXT	NULL	개선사항
additional	TEXT	NULL	추가 의견
created_at	TIMESTAMPTZ	DEFAULT NOW()	작성일

PK comments

컬럼	타입	제약조건	설명
id	UUID	PK	댓글 ID
debate_id	UUID	FK debates, NOT NULL	토론 ID
user_id	UUID	FK profiles, NOT NULL	작성자
content	TEXT	NOT NULL	댓글 내용
likes_count	INT	DEFAULT 0	좋아요 수
created_at	TIMESTAMPTZ	DEFAULT NOW()	작성일

PK NEW chat_messages

컬럼	타입	제약조건	설명
id	UUID	PK	메시지 ID
debate_id	UUID	FK debates, NOT NULL	토론(채팅방) ID
user_id	UUID	FK profiles, NOT NULL	발신자
content	TEXT	NOT NULL	메시지 내용
side	TEXT	'A' / 'B'	발신자 진영
created_at	TIMESTAMPTZ	DEFAULT NOW()	발신 시각

PK NEW notifications

컬럼	타입	제약조건	설명
id	UUID	PK	알림 ID
user_id	UUID	FK profiles, NOT NULL	수신자
type	TEXT	NOT NULL	알림 유형 (verdict/vote/comment/chat 등)
title	TEXT	NOT NULL	알림 제목
message	TEXT	NULL	알림 본문
reference_id	UUID	NULL	관련 리소스 ID (debate_id 등)
is_read	BOOLEAN	DEFAULT false	읽음 여부
created_at	TIMESTAMPTZ	DEFAULT NOW()	생성일

PK NEW debate_likes

컬럼	타입	제약조건	설명
id	UUID	PK	좋아요 ID
debate_id	UUID	FK debates, NOT NULL	토론 ID
user_id	UUID	FK profiles, NOT NULL	사용자 ID
created_at	TIMESTAMPTZ	DEFAULT NOW()	좋아요 시각

PK NEW verdict_ratings

컬럼	타입	제약조건	설명
id	UUID	PK	평가 ID
debate_id	UUID	FK debates, NOT NULL	토론 ID
user_id	UUID	FK profiles, NOT NULL	평가자
rating	INT	1~5, NOT NULL	판결 만족도 (1~5점)
created_at	TIMESTAMPTZ	DEFAULT NOW()	평가일

PK NEW page_views

컬럼	타입	제약조건	설명
id	UUID	PK	조회 ID
debate_id	UUID	FK debates, NOT NULL	토론 ID
user_id	UUID	FK profiles, NULL	조회자 (비로그인 시 NULL)
ip_address	TEXT	NULL	IP 주소
created_at	TIMESTAMPTZ	DEFAULT NOW()	조회 시각

PK NEW analytics_events

컬럼	타입	제약조건	설명
id	UUID	PK	이벤트 ID
event_type	TEXT	NOT NULL	이벤트 유형 (signup/debate_create/judgment/vote 등)
user_id	UUID	FK profiles, NULL	사용자
metadata	JSONB	NULL	이벤트 메타데이터
created_at	TIMESTAMPTZ	DEFAULT NOW()	기록일

4. 인증 API (/api/auth)

GET

/api/auth/me

AUTH

현재 사용자 프로필 조회

RESPONSE 200

```
{
  "id": "uuid",
  "nickname": "string",
  "gender": "male | female | other | null",
  "age": 25,
  "avatar_url": "string | null",
  "wins": 0, "losses": 0, "draws": 0,
  "total_score": 0, "xp": 0,
  "tier": "시민",
  "created_at": "2026-03-25T00:00:00Z"
}
```

ERROR

401	인증 토큰 없음 또는 유효하지 않음

PATCH

/api/auth/me

AUTH

프로필 수정

REQUEST BODY

```
{
  "nickname": "string (optional)",
  "avatar_url": "string (optional)"
}
```

RESPONSE 200

수정된 프로필 객체 반환 (GET /api/auth/me와 동일 형태)

POST

/api/auth/logout

AUTH

로그아웃

설명

현재 사용자의 모든 세션을 무효화합니다.

RESPONSE 200

```
{
  "message": "로그아웃 되었습니다."
}
```

5. 토론 API (/api/debates) ▼

POST

/api/debates

AUTH

토론 생성

REQUEST BODY

```
{
  "topic": "짜장면 vs 짬뽕, 어느 쪽이 더 나은 선택인가?",
  "description": "점심 메뉴 고민에서 시작된 논쟁",  // optional, 토론 설명
  "category": "일상",          // 사회 | 기술 | 정치 | 철학 | 일상 | 연애 | 문화
| 기타
  "purpose": "승부",          // 승부 | 합의 | 분석
  "lens": "일반",             // 논리 | 감정 | 현실 | 윤리 | 일반
  "mode": "duo"               // "duo" (default) | "solo" | "chat" — solo: AI 상
대, chat: 실시간 채팅
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "creator_id": "uuid",
  "opponent_id": null,
  "topic": "짜장면 vs 짬뽕...",
  "description": "점심 메뉴 고민에서 시작된 논쟁",
  "category": "일상",
  "purpose": "승부",
  "lens": "일반",
  "mode": "duo",
  "invite_code": "Ab3xK9mZ",     // 자동 생성 8자리
  "status": "waiting",            // duo: "waiting", solo: "arguing", chat: "wait
ing"
  "vote_deadline": null,
  "created_at": "2026-03-25T..."
}
```

상태 흐름: waiting → arguing → judging → voting → completed (duo/solo), waiting → chatting → judging → voting → completed (chat)

Solo 모드: `mode="solo"` 일 경우 상대방 없이 바로 `status="arguing"` 으로 시작됩니다. 사용자가 A 측 주장을 제출한 뒤 `POST /api/arguments/:debateId/solo` 를 호출하여 AI 반대 주장을 생성합니다.

GET

/api/debates

PUBLIC

토론 목록 조회

QUERY PARAMETERS

이름	타입	기본값	설명
status	String	completed	`lobby` 지정 시 waiting/chatting 상태의 chat 모드 토론 반환 (로비용)

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "category": "daily",
    "status": "completed",
    "invite_code": "Ab3xK9mZ",
    "created_at": "ISO8601",
    "creator": { "nickname": "닉네임" }
  }
]
```

> **로비 모드:** `?status=lobby` 파라미터를 사용하면 waiting/chatting 상태의 chat 모드 토론 목록을 반환합니다. 실시간 논쟁 대기실 UI에 사용됩니다.

GET

/api/debates/my/active

AUTH

내 활성 토론 조회

설명

현재 사용자가 참여 중인 활성(waiting/chatting/arguing/judging/voting) 토론 목록을 반환합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "mode": "chat",
    "status": "chatting",
    "creator": { "nickname": "..." },
    "opponent": { "nickname": "..." },
    "created_at": "ISO8601"
  }
]
```

GET

/api/debates/chat/rooms

PUBLIC

채팅방 목록 조회

설명

활성 상태(waiting/chatting)의 chat 모드 토론방 목록을 반환합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "status": "waiting",
    "mode": "chat",
    "creator": { "id": "uuid", "nickname": "...", "tier": "시민" },
    "created_at": "ISO8601"
  }
]
```

GET

/api/debates/:id

PUBLIC

토론 상세 조회

PATH PARAMS

이름	타입	설명
id	UUID	토론 ID

RESPONSE 200

```
{
  "id": "uuid",
  "topic": "string",
  "description": "string | null",
  "category": "일상",
  "purpose": "승부",
  "lens": "일반",
  "mode": "duo",
  "status": "arguing",
  "invite_code": "Ab3xK9mZ",
  "view_count": 42,
  "creator": { "id": "uuid", "nickname": "...", "tier": "시민", ... },
  "opponent": { "id": "uuid", "nickname": "...", "tier": "배심원", ... }
}
```

ERROR

404	토론을 찾을 수 없음

POST

/api/debates/join/:inviteCode

AUTH

초대 코드로 토론 참여

PATH PARAMS

이름	타입	설명
`inviteCode`	String (8자)	초대 코드

비즈니스 로직

1	invite_code + status=waiting + opponent_id=null 인 토론 검색
2	본인이 생성한 토론인지 확인 (자기 자신 참여 차단)
3	opponent_id 설정 + status를 `arguing` 으로 변경

RESPONSE 200

업데이트된 토론 객체 반환

ERROR

400	본인이 생성한 토론에 참여 시도
404	초대 코드 없음 또는 이미 진행 중인 토론

POST

/api/debates/:id/view

PUBLIC

조회수 증가

설명

토론 상세 페이지 진입 시 호출하여 view_count를 1 증가시킵니다.

PATH PARAMS

이름	타입	설명
id	UUID	토론 ID

RESPONSE 200

```
{
  "view_count": 43
}
```

DELETE

/api/debates/:id

AUTH

토론 삭제

설명

본인이 생성한 토론을 삭제합니다. CASCADE로 관련 주장/판결/투표/채팅 메시지 등 모두 삭제됩니다.

PATH PARAMS

이름	타입	설명
id	UUID	토론 ID

RESPONSE 200

```
{
  "message": "토론이 삭제되었습니다."
}
```

ERROR

403	토론 생성자가 아님
404	토론을 찾을 수 없음

6. 주장 API (/api/arguments) ▼

POST

/api/arguments/:debateId

AUTH

CONTENTFILTER

주장 제출

REQUEST BODY

```
{
  "content": "주장 내용 (50~2,000자)",
  "side": "A",              // A 또는 B
  "category": "daily",      // 콘텐츠 필터 단계 결정용
  "topic": "토론 주제"       // 게이트키퍼 주제 적합성 검사용
}
```

처리 파이프라인

```
사용자 입력
    │
    ▼
┌──────────────────────────┐
│ 1. contentFilter 미들웨어 │ ← 비속어/AI유해성/주제적합성
└──────────────────────────┘
              │ pass
              ▼
┌──────────────────────────┐
│ 2. preprocessArgument()  │ ← 인젝션 방어 + 정규화 + 유사도
│   - 2,000자 초과 → 자름  │
│   - 인젝션 패턴 → [필터링됨] │   (15개 패턴 탐지)
│   - HTML/코드블록 제거   │
│   - 공백/개행 정리       │
│   - 50자 미만 → 거부     │
│   - 상대측 유사도 85%↑ → 거부 │   (Jaccard 유사도)
└──────────────────────────┘
              │ valid
              ▼
┌──────────────────────────┐
│ 3. DB 저장 (정제된 텍스트) │
└──────────────────────────┘
```

PREPROCESSOR 상세

단계	처리	설명
인젝션 방어	15개 패턴 탐지	프롬프트 인젝션 시도를 [필터링됨]으로 치환, warnings 배열에 기록

HTML 제거	태그/코드블록 strip	HTML 태그, 마크다운 코드블록 등 제거
유사도 검사	Jaccard >85%	상대측 주장과의 토큰 기반 유사도가 85% 이상이면 거부

RESPONSE `201`

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "side": "A",
  "content": "전처리 완료된 주장 텍스트",
  "created_at": "ISO8601",
  "warnings": ["프롬프트 인젝션 시도 2건 필터링됨"]   // optional
}
```

ERROR

400	"부적절한 표현이 포함되어 있습니다" (Stage 1)
400	"유해한 콘텐츠가 감지되었습니다" (Stage 2)
400	"주제와 관련 없는 내용입니다" (Stage 3)
400	"주장 내용을 입력해주세요" (empty)
400	"주장은 50자 이상이어야 합니다" (too_short)
400	"상대측과 동일한 주장은 제출할 수 없습니다" (duplicate)

POST

/api/arguments/:debateId/solo

AUTH

솔로 모드 AI 반대 주장 생성

PATH PARAMS

이름	타입	설명
debateId	UUID	토론 ID

REQUEST BODY

없음 (요청 본문 불필요)

비즈니스 로직

1	토론 존재 여부 확인
2	토론 상태가 `arguing` 인지 확인
3	요청자가 토론 생성자(creator)인지 확인
4	A측(사용자) 주장이 이미 제출되었는지 확인
5	B측(AI) 주장이 아직 없는지 확인
6	AI가 A측 주장에 대한 반대 주장(B측)을 생성하여 저장

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": null,
  "side": "B",
  "content": "AI가 생성한 반대 주장 텍스트 (200~500자)",
  "ai_generated": true,
  "created_at": "ISO8601"
}
```

ERROR

404	"토론을 찾을 수 없습니다" (debate not found)
400	"토론이 주장 제출 단계가 아닙니다" (status != arguing)
400	"A측 주장이 먼저 필요합니다" (no A-side argument)

403	"토론 생성자만 솔로 모드를 사용할 수 있습니다" (not creator)
409	"이미 B측 주장이 존재합니다" (B-side already exists)

GET

/api/arguments/:debateId

PUBLIC

토론의 주장 목록 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "side": "A",
    "content": "주장 내용",
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임" }
  }
]
```

7. 판결 API (/api/judgments) ▼

POST

/api/judgments/:debateId

AUTH

AI 판결 요청

사전 검증

1	토론 상태가 `arguing` 인지 확인 (그 외 상태 → `400`)
2	이미 판결(verdict)이 존재하는지 확인 (중복 → `409`)
3	양측 주장(A, B)이 모두 존재하는지 확인

비즈니스 로직

1	토론 + 양측 주장(A, B) 조회
2	status → `judging` 원자적(atomic) 전환 (동시 요청 방지: arguing→judging CAS)
3	3개 AI 병렬 호출 (Promise.allSettled) — 실패 시 Grok 대체
4	개별 AI 판결을 ai_judgments 테이블에 저장
5	종합 판결 계산 (AI 점수 평균 + 다수결)
6	status → `voting` , vote_deadline = now + 24시간

GROK 폴백 메커니즘

> **Grok 대체 판결:** 3개 주요 AI(GPT-4.1, Gemini, Claude) 중 실패한 모델이 있을 경우, Grok 3 Mini를 최대 `Math.min(failedCount, 2)` 회 호출하여 대체합니다. 모든 AI(Grok 포함)가 실패하면 status를 `arguing` 으로 롤백하고 500 에러를 반환합니다.

RESPONSE `200`


```
{
  "id": "verdict-uuid",
  "debate_id": "uuid",
  "winner_side": "A",
  "summary": "A측의 구체적인 사례 제시가 결정적이었습니다...",
  "ai_score_a": 78,
  "ai_score_b": 65,
  "final_score_a": 78,
  "final_score_b": 65,
  "is_citizen_applied": false,
  "ai_judgments": [
    {
```

```
        "ai_model": "gpt-4.1",
        "winner_side": "A",
        "verdict_text": "양측의 주장을 면밀히 검토한 결과...",
        "score_a": 80, "score_b": 63,
        "score_detail_a": {
          "logic": 17, "evidence": 16, "persuasion": 15,
          "consistency": 16, "expression": 16
        },
        "score_detail_b": {
          "logic": 12, "evidence": 13, "persuasion": 13,
          "consistency": 13, "expression": 12
        },
        "confidence": 0.78
      },
      { "ai_model": "gemini-2.5-flash", "..." : "..." },
      { "ai_model": "claude-sonnet", "..." : "..." }
    ],
    "vote_deadline": "2026-03-26T00:00:00Z"
  }
```


AI 판결 5항목 루브릭 (각 0~20점, 총 100점)

항목	키	평가 기준
논리 구조	`logic`	전제→근거→결론 연결, 논증 구조의 형식적 타당성
근거 품질	`evidence`	구체적 데이터/사례/연구 인용, 출처 명시
설득력	`persuasion`	예상 반론 대응, 감정적 호소의 적절성
일관성	`consistency`	논지 일관성, 자기모순 여부
표현 적절성	`expression`	문장 명확성, 논쟁에 적합한 어조

ERROR

400	"토론이 주장 제출 단계가 아닙니다" (status != arguing)
400	"양측 주장이 모두 필요합니다" (A 또는 B 주장 미제출)
409	"이미 판결이 존재합니다" (verdict already exists)
500	모든 AI 모델 실패 (Grok 포함) — status를 arguing으로 롤백

GET

/api/judgments/:debateId

PUBLIC

판결 결과 조회

RESPONSE 200

POST 응답과 동일한 verdict + ai_judgments 객체

GET

/api/judgments/feed

PUBLIC

최근 판결 피드 (페이지네이션)

QUERY PARAMETERS

이름	타입	기본값	설명
page	Number	1	페이지 번호
limit	Number	10	페이지당 항목 수

RESPONSE 200

```
{
  "data": [
    {
      "id": "uuid",
      "winner_side": "A",
      "summary": "...",
      "ai_score_a": 78, "ai_score_b": 65,
      "final_score_a": 78, "final_score_b": 65,
      "is_citizen_applied": true,
      "debate": {
        "topic": "짜장면 vs 짬뽕",
        "category": "일상",
        "creator": { "nickname": "닉네임" }
      }
    }
  ],
  "page": 1,
  "limit": 10,
  "hasNext": true
}
```

> **페이지네이션:** `hasNext`가 `true`이면 다음 페이지가 존재합니다. 프론트엔드에서 무한 스크롤 구현 시 활용합니다.

GET

/api/judgments/daily

PUBLIC

오늘의 논쟁 목록

설명

오늘 생성된 데일리 논쟁 목록을 반환합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "오늘의 주제: ...",
    "category": "daily",
    "status": "voting",
    "created_at": "ISO8601",
    "verdict": { "winner_side": "A", "ai_score_a": 75, "ai_score_b": 68 }
  }
]
```

GET

/api/judgments/hall

PUBLIC

명예의 전당 (Hall of Fame)

설명

가장 높은 점수를 받은 판결/토론 목록을 반환합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "winner_side": "A",
    "final_score_a": 95,
    "final_score_b": 42,
    "vote_count": 120,
    "creator": { "nickname": "...", "tier": "판사" },
    "created_at": "ISO8601"
  }
]
```

POST

/api/judgments/:debateId/rate

AUTH

판결 만족도 평가 (1~5점)

REQUEST BODY

```
{
  "rating": 4    // 1~5 (필수, 정수)
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "rating": 4,
  "created_at": "ISO8601"
}
```

ERROR

400	"rating은 1~5 사이 정수여야 합니다"
404	"판결을 찾을 수 없습니다"
409	"이미 평가했습니다"

POST

/api/judgments/:debateId/retry/:model

AUTH

실패한 AI 모델 개별 재시도

설명

판결 과정에서 실패한 특정 AI 모델을 개별적으로 재시도합니다. 일부 모델만 실패했을 때 전체 판결을 다시 요청하지 않고 해당 모델만 재호출할 수 있습니다.

PATH PARAMETERS

이름	타입	설명
debateId	UUID	토론 ID
model	String	재시도할 AI 모델 (`gpt` | `gemini` | `claude` | `grok`)

RESPONSE 200

```
{
  "ai_model": "gpt-4.1",
  "winner_side": "A",
  "score_a": 72,
  "score_b": 65,
  "verdict_text": "...",
  "confidence": 0.82,
  "retried": true
}
```

ERROR

400	"유효하지 않은 모델입니다" (gpt/gemini/claude/grok 외 값)
404	"판결을 찾을 수 없습니다"
502	AI 모델 재시도 실패

8. 투표 API (/api/votes)

POST

/api/votes/:debateId

AUTH

시민 투표 (A 또는 B)

REQUEST BODY

```
{
  "voted_side": "A"    // A 또는 B (필수, 다른 값 → 400)
}
```

RESPONSE 200

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "voted_side": "A",
  "created_at": "ISO8601"
}
```

> **Upsert:** 이미 투표한 경우 투표를 변경할 수 있습니다 (마감 전까지).

ERROR

400	"voted_side는 'A' 또는 'B'만 허용됩니다" (유효성 검증)
400	"투표 기간이 아닙니다" (status != voting)
400	"투표가 마감되었습니다" (deadline 초과 → 자동 정산)

GET

/api/votes/:debateId

PUBLIC

투표 현황 조회

RESPONSE 200

```
{
  "A": 15,
  "B": 12,
  "total": 27,
  "status": "voting",
  "vote_deadline": "2026-03-26T00:00:00Z"
}
```

DELETE

/api/votes/:debateId

AUTH

투표 취소

설명

현재 사용자의 투표를 취소합니다. 투표 기간(status=voting)에만 가능합니다.

PATH PARAMETER

이름	타입	설명
debateId	UUID	토론 ID

RESPONSE 200

```
{
  "message": "투표가 취소되었습니다."
}
```

에러 응답

400	"투표 기간이 아닙니다." (status != voting)
401	인증 필요

POST

/api/votes/finalize

PUBLIC

만료된 투표 일괄 정산

비즈니스 로직 (FINALIZEVERDICT)

1	status=voting AND vote_deadline < now 인 토론 검색
2	투표 30건 이상 → 시민 점수 반영 (AI 75% + 시민 25%)
3	투표 30건 미만 → AI 점수만 사용
4	최종 승자 결정 + status → `completed`
5	양측(creator, opponent) 프로필 전적 자동 업데이트 (wins/losses/draws/total_score)

프로필 전적 자동 업데이트: 판결 확정 시 creator와 opponent의 profiles 레코드에 승/패/무 카운트 및 total_score를 자동으로 반영합니다.

RESPONSE 200

```
{
  "finalized": 3,
  "results": [
    {
      "debateId": "uuid",
      "finalA": 76, "finalB": 68,
      "finalWinner": "A",
      "citizenApplied": true,
      "voterCount": 45
    }
  ]
}
```

점수 계산: 시민 투표 30건 이상일 때 최종 = AI(25% x 3모델 = 75%) + 시민(25%). 30건 미만이면 AI 점수만 사용.

9. 프로필 API (/api/profiles)

GET

/api/profiles/:userId

PUBLIC

공개 프로필 조회

RESPONSE 200

```
{
  "id": "uuid",
  "nickname": "닉네임",
  "avatar_url": "string | null",
  "wins": 5, "losses": 3, "draws": 1,
  "total_score": 420,
  "xp": 420,
  "tier": "배심원"
}
```

GET

/api/profiles/me/verdicts

AUTH

나의 판결 기록 조회

RESPONSE 200

```
[
  {
    "id": "debate-uuid",
    "topic": "짜장면 vs 짬뽕",
    "category": "daily",
    "purpose": "compete",
    "lens": "general",
    "status": "completed",
    "created_at": "ISO8601",
    "verdicts": [
      {
        "winner_side": "A",
        "summary": "...",
        "ai_score_a": 78, "ai_score_b": 65,
        "final_score_a": 76, "final_score_b": 68,
        "is_citizen_applied": true
      }
    ]
  }
]
```

> **조건:** 사용자가 creator 또는 opponent인 completed 상태 토론만 반환

GET

/api/profiles/ranking

PUBLIC

랭킹 조회 (Top 50)

RESPONSE 200

```
[
  {
    "id": "uuid",
    "nickname": "1위유저",
    "avatar_url": "...",
    "wins": 15, "losses": 3, "draws": 2,
    "total_score": 1200,
    "xp": 1200,
    "tier": "변호사"
  }
]
```

total_score 기준 내림차순 정렬, 상위 50명. `tier` 값: 시민 / 배심원 / 변호사 / 판사 / 대법관

GET

/api/profiles/me/xp-logs

AUTH

나의 XP 이력 조회 (최근 50건)

RESPONSE 200

```
[
  {
    "id": "uuid",
    "user_id": "uuid",
    "debate_id": "uuid | null",
    "reason": "victory",
    "xp_change": 30,
    "created_at": "ISO8601"
  }
]
```

> **reason 값:** victory(+30), defeat(+5), draw(+15), daily_debate(+10), vote(+3/-3), bonus, penalty

GET

/api/profiles/me/role

AUTH

현재 사용자의 역할(관리자 여부) 조회

설명

현재 사용자의 역할(관리자 여부)을 조회합니다. 관리자 전용 UI 분기 처리에 활용됩니다.

RESPONSE 200

```
{
  "role": "user"
}
```

> **role 값:** `"user"` (일반 사용자) 또는 `"admin"` (관리자)

ERROR

401	인증 필요

DELETE

/api/profiles/me

AUTH

회원 탈퇴 (계정 삭제)

설명

현재 사용자의 계정을 삭제합니다. 프로필, 토론 참여 기록, 투표 등 모든 관련 데이터가 삭제됩니다.

RESPONSE 200

```
{
  "message": "계정이 삭제되었습니다."
}
```

ERROR

401	인증 필요

DELETE

/api/profiles/me/verdicts/:debateId

AUTH

나의 판결 기록 삭제

설명

본인이 참여한 특정 토론의 판결 기록을 삭제합니다.

PATH PARAMS

이름	타입	설명
debateId	UUID	토론 ID

RESPONSE 200

```
{
  "message": "판결 기록이 삭제되었습니다."
}
```

ERROR

403	해당 토론의 참여자가 아님
404	판결 기록을 찾을 수 없음

10. 콘텐츠 필터 API (/api/content)

POST

/api/content/check

AUTH

콘텐츠 사전 검사

REQUEST BODY

```
{
  "content": "검사할 텍스트",
  "category": "사회",          // optional, 필터 단계 결정 (한글/영문 모두 지원)
  "debateId": "uuid"           // optional, 로그 연결용
}
```

3단계 필터 기준

카테고리	Stage 1 (사전)	Stage 2 (AI 유해성)	Stage 3 (주제 적합성)
일상, 연애	O	-	-
직장, 교육	O	O	-
사회, 정치	O	O	O

RESPONSE 200

```
// 통과
{ "pass": true, "stage": 1 }

// 차단
{ "pass": false, "stage": 2, "reason": "혐오 발언 감지" }
```

GET

/api/content/logs

AUTH

내 필터 로그 조회 (최근 20건)

RESPONSE 200


```
[
  {
    "id": "uuid",
    "content_type": "argument",
    "filter_stage": 1,
    "result": "block",
    "reason": "금지어 포함: ㅅㅂ",
    "created_at": "ISO8601"
  }
]
```

11. AI 판결 시스템 ▼

AI 판사 캐릭터

AI	캐릭터	모델	성격	말투
Judge G	분석적 판사	GPT-4.1	균형잡힌, 분석적, 정중한	격식체 (~입니다, ~하겠습니다)
Judge M	통찰적 판사	Gemini 2.5 Flash	다각적, 창의적, 통찰력	친근체 (~거든요, ~잖아요)
Judge C	배려하는 판사	Claude Sonnet 4	신중한, 공정한, 배려하는	부드러운 존댓말 (~네요, ~것 같아요)
Judge X	대체 판사	Grok 3 Mini	Judge G 스타일	격식체 (폴백용)

판결 프롬프트 구조 (System / User 분리)

```
┌──────────────────────────────────────────────┐
│                System Prompt                 │
│  ┌──────────────────────────────────────┐    │
│  │ 캐릭터 설정 (Judge G/M/C 고유 성격)    │    │
│  │ + 판결문 톤 예시                       │    │
│  │ + 판결 원칙 (6개)                      │    │
│  │ + 5항목 루브릭 (각 20점)               │    │
│  │ + confidence 기준표 (0.50~1.00)        │    │
│  │ + 금지 사항 (6개)                      │    │
│  │ + JSON 응답 스키마                     │    │
│  └──────────────────────────────────────┘    │
├──────────────────────────────────────────────┤
│                User Prompt                   │
│  ┌──────────────────────────────────────┐    │
│  │ 논쟁 주제 (topic)                      │    │
│  │ + 판결 목적 (purpose)                  │    │
│  │ + 분석 렌즈 (lens + 가중치 설명)       │    │
│  │ + A측 주장 원문                        │    │
│  │ + B측 주장 원문                        │    │
│  └──────────────────────────────────────┘    │
└──────────────────────────────────────────────┘
```

판결 목적 (purpose)

값	설명	판결 방식
`compete`	승부 판별	논리적 우열 → 승/패/무 판정 + 점수
`consensus`	합의 도출	양측 공통 접점과 합리적 결론 도출
`analysis`	분석 요청	양측 논리를 객관적으로 정리 비교

분석 렌즈 (lens) — 가중치 조정

값	설명	가중 항목
`general`	종합 관점	5개 항목 균등 (각 20점)
`logic`	논리/팩트	logic 항목에 가중치
`emotion`	관계/감정	persuasion 항목에 가중치
`practical`	실용/비용	evidence 항목에 가중치
`ethics`	윤리/가치	consistency 항목에 가중치

confidence 기준표

구간	의미	기준
0.90 ~ 1.00	압도적 차이	한쪽 구체적 근거 다수 vs 상대측 근거 전무
0.80 ~ 0.89	명확한 차이	양측 근거 있으나 한쪽 논리 구조가 확실히 우수
0.70 ~ 0.79	분명한 차이	양측 유사 수준이나 핵심 논점에서 한쪽 우위
0.55 ~ 0.69	근소한 차이	양측 모두 우수하나 미세한 차이로 판정
0.50 ~ 0.54	사실상 동점	draw 판정 시에만 사용

AI API 설정

항목	GPT-4.1	Gemini 2.5 Flash	Claude Sonnet 4	Grok 3 Mini
temperature	0.3	0.3	-	0.3
max_tokens	-	-	1,024	-
timeout	30초	30초	30초	30초
JSON 강제	response_format	responseMimeType	regex 파싱	response_format
prompt 분리	system + user	systemInstruction + contents	system + messages	system + user

Solo 모드 — AI 반대 주장 생성

항목	설정
사용 모델	GPT-4.1 (단독)
생성 길이	200~500자
temperature	0.7 (다양한 반대 의견)
저장	arguments 테이블, user_id: null, ai_generated: true

엔드포인트	`POST /api/arguments/:debateId/solo`
판결 시 공개	AI 생성 사실 비공개 (공정성)

12. 피드백 API (/api/feedbacks)

POST

/api/feedbacks

AUTH

피드백 제출 (Upsert)

설명

서비스 만족도 피드백을 제출합니다. 기존 피드백이 있으면 수정(upsert), 없으면 새로 생성합니다.

REQUEST BODY

```
{
  "satisfaction": 4.5,     // 0.5~5 (0.5 단위, 필수)
  "ai_accuracy": 4.0,     // 0.5~5 (0.5 단위, 필수)
  "ui_ease": 3.5,         // 0.5~5 (0.5 단위, 필수)
  "fairness": 4.0,        // 0.5~5 (0.5 단위, 필수)
  "recommend": 5.0,       // 0.5~5 (0.5 단위, 필수)
  "best_feature": "AI 판결이 재밌어요",  // optional (텍스트)
  "improvement": "로딩이 느려요",        // optional (텍스트)
  "additional": "추가 의견"              // optional (텍스트)
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "user_id": "uuid",
  "satisfaction": 4.5,
  "ai_accuracy": 4.0,
  "ui_ease": 3.5,
  "fairness": 4.0,
  "recommend": 5.0,
  "best_feature": "AI 판결이 재밌어요",
  "improvement": "로딩이 느려요",
  "additional": "추가 의견",
  "created_at": "ISO8601"
}
```

에러 응답

400	"satisfaction는 0.5~5 사이 0.5 단위 값이어야 합니다." (유효성 검증)
401	인증 필요

GET

/api/feedbacks/me

AUTH

나의 피드백 조회

RESPONSE 200


```
[
  {
    "id": "uuid",
    "satisfaction": 4.5,
    "ai_accuracy": 4.0,
    "ui_ease": 3.5,
    "fairness": 4.0,
    "recommend": 5.0,
    "best_feature": "...",
    "improvement": "...",
    "additional": "...",
    "created_at": "ISO8601"
  }
]
```

13. OG 메타 API (/og)

GET

/og/invite/:inviteCode

PUBLIC

동적 OG 메타태그 (초대 링크 공유용)

설명

초대 코드로 토론 정보를 조회하여 동적 OG 메타태그가 포함된 HTML을 반환합니다. 카카오톡/Discord/슬랙 등에서 링크 미리보기(OG 카드)에 토론 주제가 표시됩니다. 브라우저 접속 시 `<meta http-equiv="refresh">`로 클라이언트 초대 페이지로 자동 리다이렉트됩니다.

PATH PARAMETER

이름	타입	설명
inviteCode	String	초대 코드 (8자리)

RESPONSE 200 (TEXT/HTML)

```
<!DOCTYPE html>
<html>
<head>
  <meta property="og:title" content="모라고라 - {토론 주제}" />
  <meta property="og:description" content="{카테고리} | {목적} 논쟁에 초대합니다"
/>
  <meta property="og:image" content="https://team-moragora-client.vercel.app/og-c
ard.png" />
  <meta http-equiv="refresh" content="0;url=https://team-moragora-client.vercel.a
pp/invite/{inviteCode}" />
</head>
</html>
```

> **사용법:** 공유 링크를 `https://teammoragora.onrender.com/og/invite/{inviteCode}` 로 전달하면, 크롤러(카카오/디스코드)는 OG 태그를 읽고, 브라우저 사용자는 클라이언트 페이지로 자동 이동합니다.

14. 채팅 API (/api/chat) [NEW]

GET

/api/chat/:debateId/messages

AUTH

채팅 메시지 내역 조회

설명

특정 토론(채팅방)의 전체 메시지 내역을 시간순으로 반환합니다.

PATH PARAMS

이름	타입	설명
debateId	UUID	토론(채팅방) ID

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "content": "메시지 내용",
    "side": "A",
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임", "avatar_url": "..." }
  }
]
```

POST

/api/chat/:debateId/messages

AUTH

채팅 메시지 전송

REQUEST BODY

```
{
  "content": "메시지 내용",
  "side": "A"    // A 또는 B
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "content": "메시지 내용",
  "side": "A",
  "created_at": "ISO8601"
}
```

> **참고:** 실시간 채팅은 주로 Socket.io를 통해 처리됩니다. 이 REST API는 메시지 영구 저장 및 폴백용으로 사용됩니다.

POST

/api/chat/:debateId/start

AUTH

채팅 세션 시작

설명

채팅 모드 토론의 실시간 대화 세션을 시작합니다. 토론 상태가 `chatting` 으로 전환됩니다.

RESPONSE 200

```
{
  "message": "채팅이 시작되었습니다.",
  "debate_id": "uuid",
  "status": "chatting"
}
```

ERROR

400	"채팅을 시작할 수 없는 상태입니다"
403	"토론 참여자만 채팅을 시작할 수 있습니다"

POST

/api/chat/:debateId/end

AUTH

채팅 종료 및 판결 요청

설명

채팅 세션을 종료하고 채팅 내역을 기반으로 AI 판결을 트리거합니다. 채팅 메시지가 각 진영의 주장으로 변환되어 판결에 사용됩니다.

RESPONSE 200

```
{
  "message": "채팅이 종료되었습니다. 판결이 진행됩니다.",
  "debate_id": "uuid",
  "status": "judging"
}
```

ERROR

400	"채팅 중인 토론만 종료할 수 있습니다" (status != chatting)
403	"토론 참여자만 채팅을 종료할 수 있습니다"

15. 댓글 API (/api/comments) [NEW]

GET

/api/comments/:debateId

PUBLIC

토론 댓글 목록 조회

RESPONSE 200


```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "content": "댓글 내용",
    "likes_count": 5,
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임", "avatar_url": "...", "tier": "배심원" }
  }
]
```

POST

/api/comments/:debateId

AUTH

댓글 작성

REQUEST BODY

```
{
  "content": "댓글 내용 (1~500자)"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "content": "댓글 내용",
  "likes_count": 0,
  "created_at": "ISO8601"
}
```

ERROR

400	"댓글 내용을 입력해주세요"

DELETE

/api/comments/:commentId

AUTH

댓글 삭제 (본인만)

RESPONSE 200

```
{
  "message": "댓글이 삭제되었습니다."
}
```

ERROR

403	"본인의 댓글만 삭제할 수 있습니다"
404	"댓글을 찾을 수 없습니다"

POST

/api/comments/:commentId/like

AUTH

댓글 좋아요 토글

설명

댓글에 좋아요를 누르거나 취소합니다 (토글). 이미 좋아요한 상태에서 다시 호출하면 취소됩니다.

RESPONSE 200

```
{
  "liked": true,       // true: 좋아요 추가, false: 좋아요 취소
  "likes_count": 6
}
```

16. 알림 API (/api/notifications) [NEW]

GET

/api/notifications

AUTH

알림 목록 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "type": "verdict",
    "title": "판결이 완료되었습니다",
    "message": "짜장면 vs 짬뽕 논쟁의 판결 결과를 확인하세요.",
    "reference_id": "debate-uuid",
    "is_read": false,
    "created_at": "ISO8601"
  }
]
```

GET

/api/notifications/unread-count

AUTH

읽지 않은 알림 수 조회

RESPONSE 200

```
{
  "count": 3
}
```

PATCH

/api/notifications/:id/read

AUTH

알림 읽음 처리

RESPONSE 200

```
{
  "id": "uuid",
  "is_read": true
}
```

PATCH

/api/notifications/read-all

AUTH

모든 알림 읽음 처리

RESPONSE 200

```
{
  "message": "모든 알림을 읽음 처리했습니다.",
  "updated": 5
}
```

DELETE

/api/notifications/:id

AUTH

알림 삭제

RESPONSE 200

```
{
  "message": "알림이 삭제되었습니다."
}
```

DELETE

/api/notifications

AUTH

모든 알림 삭제

RESPONSE 200

```
{
  "message": "모든 알림이 삭제되었습니다.",
  "deleted": 12
}
```

17. AI 생성 API (/api/ai) [NEW]

POST

/api/ai/generate-sides

PUBLIC

주제에서 찬반 입장 자동 생성

설명

사용자가 입력한 논쟁 주제를 분석하여 찬성(A측)과 반대(B측) 입장을 AI가 자동으로 생성합니다. 토론 생성 시 주장 작성 보조 용도로 사용됩니다.

REQUEST BODY

```
{
  "topic": "재택근무가 사무실 근무보다 나은가?"
}
```

RESPONSE 200

```
{
  "side_a": "재택근무가 더 효율적이다",
  "side_b": "사무실 근무가 더 효율적이다",
  "description": "업무 환경에 따른 생산성과 워라밸 비교"
}
```

ERROR

400	"주제를 입력해주세요"
502	AI 서비스 호출 실패

POST

/api/ai/analyze-topic

PUBLIC

AI가 논쟁 주제를 분석하여 핵심 쟁점 도출

설명

사용자가 입력한 논쟁 주제를 AI가 분석하여 핵심 쟁점과 논점을 도출합니다. 토론 생성 전 주제 탐색 용도로 사용됩니다.

REQUEST BODY

```
{
  "topic": "재택근무가 사무실 근무보다 나은가?"
}
```

RESPONSE 200

```
{
  "analysis": "이 주제의 핵심 쟁점은 1) 생산성 차이, 2) 워라밸과 삶의 질, 3) 팀 협업과 소통 효율성, 4) 비용 구조(통근비 vs 홈오피스 비용)입니다. 찬성 측은 유연성과 자율성을, 반대 측은 대면 소통과 조직문화를 강조할 수 있습니다."
}
```

ERROR

400	"주제를 입력해주세요"
502	AI 서비스 호출 실패

POST

/api/ai/socratic-feedback

AUTH

소크라테스 코칭 — AI가 주장의 논리적 허점을 질문으로 피드백

설명

사용자가 작성 중인 주장을 AI가 소크라테스식 질문법으로 분석하여, 논리적 허점이나 보완할 수 있는 포인트를 질문 형태로 피드백합니다. 주장 제출 전 논증 품질을 높이기 위한 코칭 용도입니다.

REQUEST BODY

```
{
  "topic": "재택근무가 사무실 근무보다 나은가?",
  "side": "A",
  "argument": "재택근무는 출퇴근 시간을 절약하여 생산성을 높인다. 또한 자유로운 환경에서 창의력이 증가한다."
}
```

RESPONSE 200

```
{
  "questions": [
    "출퇴근 시간 절약이 곧 생산성 향상으로 이어진다는 근거가 있나요? 절약된 시간을 실제로 업무에 사용한다는 데이터가 있을까요?",
    "자유로운 환경이 모든 직종에서 창의력을 높이나요? 반대로 구조화된 환경이 더 효과적인 경우는 없을까요?",
    "재택근무의 단점(고립감, 소통 비용 등)에 대한 반론은 어떻게 준비하시겠어요?"
  ],
  "strength": "출퇴근 시간 절약이라는 구체적 장점을 제시한 점이 좋습니다.",
  "suggestion": "구체적인 통계나 연구 결과를 인용하면 설득력이 크게 향상됩니다."
}
```

ERROR

400	"주장을 입력해주세요" (argument 누락 또는 빈 값)
400	"주제를 입력해주세요" (topic 누락)
401	인증 필요
502	AI 서비스 호출 실패

POST

/api/ai/rubric-score

AUTH

루브릭 점수 — AI가 주장을 5항목 루브릭으로 사전 채점

설명

사용자가 작성 중인 주장을 판결에 사용되는 5항목 루브릭(논리 구조, 근거 품질, 설득력, 일관성, 표현 적절성)으로 사전 채점합니다. 제출 전 자신의 주장 품질을 미리 확인하고 개선할 수 있습니다.

REQUEST BODY

```
{
  "topic": "재택근무가 사무실 근무보다 나은가?",
  "side": "A",
  "argument": "재택근무는 출퇴근 시간을 절약하여 생산성을 높인다. 스탠퍼드 연구에 따르면 재택근무자의 생산성이 13% 높았다. 또한 자유로운 환경에서 창의력이 증가한다."
}
```

RESPONSE 200

```
{
  "total_score": 68,
  "score_detail": {
    "logic": 14,
    "evidence": 15,
    "persuasion": 13,
    "consistency": 14,
    "expression": 12
  },
  "feedback": {
    "logic": "전제→근거→결론 흐름은 잡혀 있으나, '창의력 증가'와 '생산성 향상' 사이의 논리적 연결이 부족합니다.",
    "evidence": "스탠퍼드 연구 인용이 좋습니다. 창의력 관련 구체적 데이터도 추가하면 더 강력해집니다.",
    "persuasion": "반대측 예상 반론(소통 비용, 고립감)에 대한 선제 대응이 없습니다.",
    "consistency": "전반적으로 일관성 있으나, 생산성과 창의력이라는 두 축을 더 명확히 구분하세요.",
    "expression": "문장이 간결하나, 좀 더 구체적인 표현과 수치를 활용하면 좋겠습니다."
  },
  "grade": "B"
}
```

등급 기준: S(90~100), A(80~89), B(70~79), C(60~69), D(50~59), F(0~49)

ERROR

400	"주장을 입력해주세요" (argument 누락 또는 빈 값)
400	"주제를 입력해주세요" (topic 누락)
401	인증 필요
502	AI 서비스 호출 실패

18. Cron API (/api/cron) [NEW]

인증: Cron API는 `CRON_SECRET` 헤더 검증을 통해 보호됩니다. 외부 스케줄러(Vercel Cron, cron-job.org 등)에서만 호출해야 합니다.

POST

/api/cron/daily-debate

SECRET

데일리 논쟁 자동 생성

설명

매일 자동으로 "오늘의 논쟁"을 생성합니다. AI가 시사적이고 흥미로운 주제를 선정하여 토론을 생성합니다.

REQUEST HEADER

```
Authorization: Bearer {CRON_SECRET}
```

RESPONSE 201

```
{
  "id": "uuid",
  "topic": "AI가 생성한 오늘의 논쟁 주제",
  "category": "daily",
  "status": "arguing",
  "created_at": "ISO8601"
}
```

DELETE

/api/cron/daily-debate/:id

SECRET

데일리 논쟁 삭제

설명

특정 데일리 논쟁을 삭제합니다. 잘못 생성된 데일리 논쟁 정리용입니다.

RESPONSE 200

```
{
  "message": "데일리 논쟁이 삭제되었습니다."
}
```

19. 관리자 API (/api/admin) [NEW]

인증: Admin API는 관리자 권한이 필요합니다. 일반 사용자는 접근할 수 없습니다.

GET

/api/admin/stats

ADMIN

대시보드 통계

RESPONSE 200

```
{
  "total_users": 1250,
  "total_debates": 3400,
  "total_verdicts": 2800,
  "total_votes": 15000,
  "active_debates": 45,
  "today_signups": 12,
  "today_debates": 28
}
```

GET

/api/admin/ai

ADMIN

AI 모델 통계

RESPONSE 200


```
{
  "models": [
    { "name": "gpt-4.1", "total_calls": 2800, "failures": 12, "avg_latency_ms": 4500 },
    { "name": "gemini-2.5-flash", "total_calls": 2800, "failures": 8, "avg_latency_ms": 3200 },
    { "name": "claude-sonnet", "total_calls": 2800, "failures": 15, "avg_latency_ms": 5100 },
    { "name": "grok-3-mini", "total_calls": 35, "failures": 2, "avg_latency_ms": 3800 }
  ],
  "fallback_rate": 0.012
}
```


GET

/api/admin/trends

ADMIN

일별 트렌드

RESPONSE 200


```
[
  {
    "date": "2026-03-25",
    "signups": 12,
    "debates_created": 28,
    "verdicts_completed": 22,
    "votes_cast": 180,
    "active_users": 95
  }
]
```

GET

/api/admin/analytics

ADMIN

분석 데이터

RESPONSE 200


```
{
  "retention": { "d1": 0.45, "d7": 0.32, "d30": 0.18 },
  "popular_categories": [
    { "category": "일상", "count": 1200 },
    { "category": "연애", "count": 850 },
    { "category": "사회", "count": 620 }
  ],
  "avg_verdict_rating": 4.2,
  "avg_messages_per_chat": 24,
  "conversion_rate": { "signup_to_debate": 0.65, "debate_to_verdict": 0.82 }
}
```

20. Socket.io 이벤트 [NEW]

연결: Socket.io 서버는 Express 서버와 동일한 포트(5000)에서 실행됩니다. 클라이언트는 `io(SERVER_URL)` 로 연결합니다. 인증은 연결 시 `auth.token` 으로 JWT를 전달합니다.

```
// 클라이언트 연결 예시
const socket = io(SERVER_URL, {
  auth: { token: supabase_access_token }
});
```

Client → Server (emit)

EMIT

join-room

CLIENT→SERVER

채팅방 입장

PAYLOAD

```
{
  "debateId": "uuid"
}
```

특정 토론 채팅방에 입장합니다. 해당 방의 메시지를 수신하기 시작합니다.

EMIT

leave-room

CLIENT→SERVER

채팅방 퇴장

PAYLOAD

```
{
  "debateId": "uuid"
}
```

EMIT

join-presence

CLIENT→SERVER

프레즌스(대기실) 입장

PAYLOAD

```
{
  "debateId": "uuid",
  "userId": "uuid",
  "nickname": "닉네임"
}
```

대기실에 입장하여 상대방의 접속 상태를 실시간으로 확인합니다.

EMIT

select-side

CLIENT→SERVER

진영 선택

PAYLOAD

```
{
  "debateId": "uuid",
  "side": "A"    // A 또는 B
}
```

EMIT

leave-presence

CLIENT→SERVER

프레즌스(대기실) 퇴장

PAYLOAD

```
{
  "debateId": "uuid"
}
```

EMIT

typing

CLIENT→SERVER

타이핑 상태 전송

PAYLOAD

```
{
  "debateId": "uuid",
  "isTyping": true
}
```

상대방에게 타이핑 중임을 알립니다. `isTyping: false` 로 타이핑 종료를 알립니다.

EMIT

start-game

CLIENT→SERVER

실시간 논쟁 시작

PAYLOAD

```
{
  "debateId": "uuid"
}
```

양측이 모두 준비되면 실시간 논쟁(채팅)을 시작합니다. 서버에서 타이머가 시작됩니다.

EMIT

send-message

CLIENT→SERVER

채팅 메시지 전송

PAYLOAD

```
{
  "debateId": "uuid",
  "content": "메시지 내용",
  "side": "A"
}
```

채팅 메시지를 전송합니다. 서버에서 DB 저장 후 방의 모든 참여자에게 브로드캐스트합니다. 토론 상태가 waiting일 경우 자동으로 chatting으로 전환되며 서버 타이머가 시작됩니다.

EMIT

request-time-change

CLIENT→SERVER

토론 시간 변경 요청

PAYLOAD

```
{
  "debateId": "uuid",
  "newTime": 600    // 초 단위 (예: 600 = 10분)
}
```

상대방에게 토론 시간 변경을 제안합니다. 상대방이 수락하면 적용됩니다.

EMIT

vote-time-change

CLIENT→SERVER

시간 변경 투표 (수락/거절)

PAYLOAD

```
{
  "debateId": "uuid",
  "approved": true    // true: 수락, false: 거절
}
```

Server → Client (on)

ON

presence-sync

SERVER→CLIENT

대기실 참여자 상태 동기화

PAYLOAD

```
{
  "users": [
    { "userId": "uuid", "nickname": "닉네임", "side": "A", "online": true },
    { "userId": "uuid", "nickname": "상대방", "side": "B", "online": true }
  ]
}
```

대기실에 참여 중인 사용자 목록과 진영 선택 상태를 동기화합니다.

ON

opponent-typing

SERVER→CLIENT

상대방 타이핑 상태

PAYLOAD

```
{
  "isTyping": true
}
```

ON

opponent-returned

SERVER→CLIENT

상대방 복귀

상대방이 연결 해제 후 다시 접속했을 때 발생합니다.

ON

opponent-left

SERVER→CLIENT

상대방 이탈

상대방이 채팅방에서 연결이 끊어졌을 때 발생합니다.

ON

game-start

SERVER→CLIENT

논쟁 시작 알림

PAYLOAD

```
{
  "debateId": "uuid",
  "timeLimit": 300,    // 초 단위
  "startedAt": "ISO8601"
}
```

양측 준비 완료 후 논쟁이 시작되었음을 알립니다. 클라이언트에서 카운트다운 타이머를 시작합니다.

ON

new-message

SERVER→CLIENT

새 채팅 메시지 수신

PAYLOAD

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "content": "메시지 내용",
  "side": "A",
  "created_at": "ISO8601",
  "user": { "nickname": "닉네임" }
}
```

ON

chat-ended

SERVER→CLIENT

채팅 종료 (참여자에 의해)

PAYLOAD

```
{
  "debateId": "uuid",
  "endedBy": "uuid",    // 종료한 사용자 ID
  "reason": "manual"
}
```

참여자 중 한 명이 채팅을 종료했을 때 발생합니다. 이후 AI 판결이 진행됩니다.

ON

chat-auto-ended

SERVER→CLIENT

채팅 자동 종료 (타이머 만료)

PAYLOAD

```
{
  "debateId": "uuid",
  "reason": "timeout"
}
```

설정된 토론 시간이 만료되어 채팅이 자동 종료되었을 때 발생합니다.

ON

filter-blocked

SERVER→CLIENT

메시지 필터 차단 알림

PAYLOAD

```
{
  "reason": "부적절한 표현이 포함되어 있습니다."
}
```

전송한 메시지가 콘텐츠 필터에 의해 차단되었을 때 발신자에게만 전달됩니다.

ON

time-change-request

SERVER→CLIENT

시간 변경 요청 수신

PAYLOAD

```
{
  "requestedBy": "uuid",
  "nickname": "닉네임",
  "newTime": 600    // 초 단위
}
```

상대방이 토론 시간 변경을 요청했을 때 발생합니다. UI에서 수락/거절 버튼을 표시합니다.

ON

time-change-approved

SERVER→CLIENT

시간 변경 승인

PAYLOAD

```
{
  "newTime": 600,
  "approvedBy": "uuid"
}
```

시간 변경 요청이 승인되어 새 시간이 적용되었을 때 발생합니다.

ON

time-change-cancelled

SERVER→CLIENT

시간 변경 거절

PAYLOAD

```
{
  "cancelledBy": "uuid",
```

```
    "reason": "rejected"
  }
```

시간 변경 요청이 거절되었을 때 발생합니다.

21. 에러 코드 정리

코드	의미	발생 상황
200	OK	정상 조회, 업데이트 성공
201	Created	토론/주장/투표/댓글/메시지 생성 성공
400	Bad Request	콘텐츠 필터 차단, 유효성 검증 실패, 토론 상태 불일치, 투표 마감 초과, voted_side 유효성
401	Unauthorized	JWT 토큰 없음 또는 만료
403	Forbidden	권한 없음 (Solo 모드: 생성자가 아닌 사용자, 타인 댓글 삭제, 관리자 API 접근)
404	Not Found	토론/사용자/초대코드/댓글/알림 없음
409	Conflict	중복 판결 요청, Solo 모드 B측 주장 중복, 중복 평가
500	Internal Error	DB 오류, 모든 AI 모델 실패 (status → arguing 롤백)
502	Bad Gateway	AI 서비스 호출 실패

공통 에러 응답 형식

```
{
  "error": "에러 메시지 (한국어)"
}
```